StockKeep

Shop Inventory & Stock Tracking System

Project Documentation

Project_Documentation.pdf — Master Project Report

Student Name:	[STUDENT NAME — TO BE PROVIDED]
Student ID:	[STUDENT ID — TO BE PROVIDED]
Project Title:	StockKeep — Shop Inventory & Stock Tracking System
Institution:	[INSTITUTION NAME — TO BE PROVIDED]
Department:	[DEPARTMENT NAME — TO BE PROVIDED]
Module / Course:	[MODULE / COURSE CODE — TO BE PROVIDED]
Supervisor:	[SUPERVISOR NAME — TO BE PROVIDED]
Date:	13 August 2026

This is the master project report and is intended to be read alongside five standalone companion documents in this submission package: *SRS.pdf*, *Testing_Report.pdf*, *Technical_Debt_Plan.pdf*, *User_Manual.pdf*, and *Deployment_and_Source_Links.txt*. All five are cross-referenced from the relevant chapters below and are consistent with this report. This documentation was prepared by directly inspecting the StockKeep source code repository and, where stated, by executing live tests against a running instance of the application; claims that could not be verified this way are explicitly marked rather than assumed.

Contents

List of Figures

List of Tables

1. Project Title

StockKeep — Shop Inventory & Stock Tracking System, an AI-assisted web application for small-shop inventory, stock movement, sales, and supplier management.

2. Background

Small, single-location shops frequently manage stock using spreadsheets, paper logs, or memory. This creates avoidable problems: stock discrepancies go unnoticed until a customer is turned away or capital sits in slow-moving inventory, and no one has real-time visibility of what is actually on the shelf. StockKeep was built as a lightweight, self-contained alternative purpose-built for a single shop with a small staff team, rather than an enterprise-scale ERP/POS system that would be disproportionately complex to deploy and operate for that context.

3. Problem Statement

A small shop needs a simple, fast, always-available way to: know what stock it has, know when it is running low, record what leaves the shelf (whether sold or otherwise), and get a quick read on how the business is performing — without requiring dedicated IT staff, a large upfront cost, or a lengthy onboarding process for staff.

4. Aim

To design, build, and document a functional, deployable, single-shop inventory and stock-tracking web application that supports the core stock lifecycle (add, adjust, sell) with a real-time dashboard and optional AI-assisted insights, and to produce academic documentation for it that is fully traceable to the delivered code and to live test evidence.

5. Objectives

1. Implement passcode-gated access to the system (FR-01–FR-04).
2. Implement full inventory item lifecycle management: create, read, update, delete, and search/filter (FR-06–FR-10).
3. Implement auditable stock movements (Stock-In/Stock-Out) with a hard business rule preventing negative stock (FR-11–FR-13).
4. Implement sales recording that automatically reconciles against stock (FR-15).
5. Implement supplier management (FR-16).
6. Provide a real-time dashboard, inventory-value/dead-stock reports, and sales analytics (FR-05, FR-17).
7. Integrate an AI assistant (chat, item descriptions, stockout forecasting, dashboard insights) that degrades gracefully to deterministic logic if the AI provider is unavailable (FR-19–FR-22).
8. Support zero-configuration first-run demo data so the system is immediately explorable (FR-23).
9. Produce complete, evidence-based academic documentation (this package) rather than documentation written independently of the actual delivered system.

6. Stakeholders

Table 1 — Stakeholders and Interests

Stakeholder	Interest
Shop owner/manager	Accurate, real-time visibility of stock levels, sales, and reorder needs
Shop staff (end user)	A fast, simple interface to record stock-in/out and sales during a shift
Student developer	Deliver a working, demonstrable, and academically documented system
Module supervisor/examiner	Assess the system and documentation against the marking rubric
Suppliers (indirect)	Represented as data records only; not system users

7. Requirements Analysis

Requirements were derived directly from the implemented feature set by inspecting every route under `src/app/api/**/route.ts`, every page under `src/app/`, and the Prisma schema, rather than written speculatively ahead of the build. The full requirements set, including source-file traceability, system interfaces, database/authentication/security requirements, constraints, assumptions, and acceptance criteria, is maintained in the standalone **`SRS.pdf`** and summarised in the next two sections and Section 10.

8. Functional Requirements

Table 2 — Functional Requirements Summary (full detail with file-level traceability in `SRS.pdf`, Section 2)

ID range	Area
FR-01–FR-04	Authentication, session, logout
FR-05	Dashboard & analytics
FR-06–FR-10	Inventory item CRUD, search/filter, detail view
FR-11–FR-14	Stock adjustment, zero-floor rule, atomic transaction, low-stock email
FR-15	Sales recording with stock reconciliation
FR-16	Supplier CRUD
FR-17	Inventory/dead-stock reporting
FR-18	Store settings
FR-19–FR-22	AI chat, description, forecast, insights (each with deterministic fallback)
FR-23	Auto-seed sample data

9. Non-Functional Requirements

Table 3 — Non-Functional Requirements Summary (full detail in `SRS.pdf`, Section 3)

Category	Status
Security (route protection)	Implemented
Security (passcode hashing)	**Not met** — TD-01
Reliability (AI graceful degradation)	Implemented and confirmed by live testing
Data integrity (atomic writes, non-negative stock)	Implemented and confirmed by live testing
Maintainability (TypeScript strict mode)	Implemented
Portability (Vercel deployability)	Implemented, with a persistence caveat — TD-08
Usability (responsive layout)	Styled for it; **not verified** by device testing
Performance	**Not benchmarked**
Scalability (database)	**At risk** — TD-08
Testability (automated coverage)	**Not met** — 0% automated coverage, TD-07

10. SRS

The complete Software Requirements Specification — introduction, purpose, scope, product perspective, user classes, full functional/non-functional requirements, system interfaces, database/authentication/security requirements, constraints, assumptions, acceptance criteria, prioritisation, and traceability — is provided as the standalone document `SRS.pdf`, included in this submission package. It is written to be fully consistent with this master report.

11. Requirements Prioritisation

Table 4 — Requirements Prioritisation (MoSCoW)

Priority	Requirements
Must have	Auth (FR-01–04), core inventory CRUD + stock adjustment (FR-06–13), sales (FR-15)
Should have	Dashboard (FR-05), suppliers/reports/settings (FR-16–18), AI graceful degradation
Could have	AI chat/describe/forecast/insights (FR-19–22)
Won't have (this version)	Multi-user roles, barcode scanning, multi-store support, automated test suite

12. Software Effort Estimation

Table 5 — Software Effort Estimate

Work item	Estimated effort
Data model + Prisma schema design	0.5 day
Auth + middleware	0.5 day
Inventory CRUD + validation	1 day
Stock adjustment + sales transactional logic	1 day
Dashboard, analytics, reports endpoints	1 day
AI integration (chat, describe, forecast, insights) + fallback logic	1.5 days
Frontend SPA shell (all 9 views)	1.5 days
Deployment hardening (Vercel/Prisma build fixes, auto-seed)	0.5 day
Total implementation	**~7.5 days**
This documentation package (analysis, diagrams, live testing, 6 documents)	~1 day (separate, produced after the fact)

This estimate is reconstructed from the shape and sequencing of the Git history (a substantial initial commit followed by four focused hardening commits) rather than from timesheets, since none were kept; it should be treated as indicative, not authoritative. **Not verified against actual developer time logs — execution required by student** if precise effort accounting is required by the rubric.

13. System Analysis

StockKeep is a single-tenant, single-role system: one shop, one shared credential, no distinct user accounts. The dominant data flow is a request/response cycle from the browser to a Next.js API route, through validation, into Prisma/SQLite, and back — with two optional external calls (Gemini, Resend) that never block the primary flow due to fallback design. The system was analysed by tracing every API route and its Prisma calls, confirmed with live HTTP testing (see `Testing_Report.pdf`) rather than by reading source code alone.

14. System Design

Design decisions and their rationale:

- **Framework**: Next.js 16 App Router was chosen to co-locate frontend and backend (API routes) in a single deployable unit, appropriate for a small, single-developer project on a short timeline.
- **ORM**: Prisma was chosen for its type-safe query builder (eliminating a class of SQL-injection risk by construction — confirmed by code review, no raw SQL calls exist in the codebase) and for its migration/schema tooling.
- **Database**: SQLite was chosen for zero-configuration local development; this decision is flagged as a production risk (TD-08) given the Vercel serverless deployment target.
- **Validation**: Zod schemas for the two highest-risk write paths (item creation/update, stock adjustment); simpler manual checks elsewhere. This is an intentional (if inconsistent) trade-off given project time constraints, formally logged rather than hidden — see `Technical_Debt_Plan.pdf`.
- **AI integration**: every AI-touching route computes a deterministic answer *first* (or wraps the AI call in try/catch) so that AI unavailability is never a user-facing failure — verified live in `Testing_Report.pdf` (TC-API-20, TC-API-21).

15. Architecture

StockKeep follows a three-tier architecture: a React 19 client tier, a Next.js serverless application tier (middleware, API routes, validation, AI layer), and a data tier (SQLite plus two external APIs). See **Figure A1**.

Figure A1 — StockKeep System Architecture (3-Tier, Next.js Full-Stack)

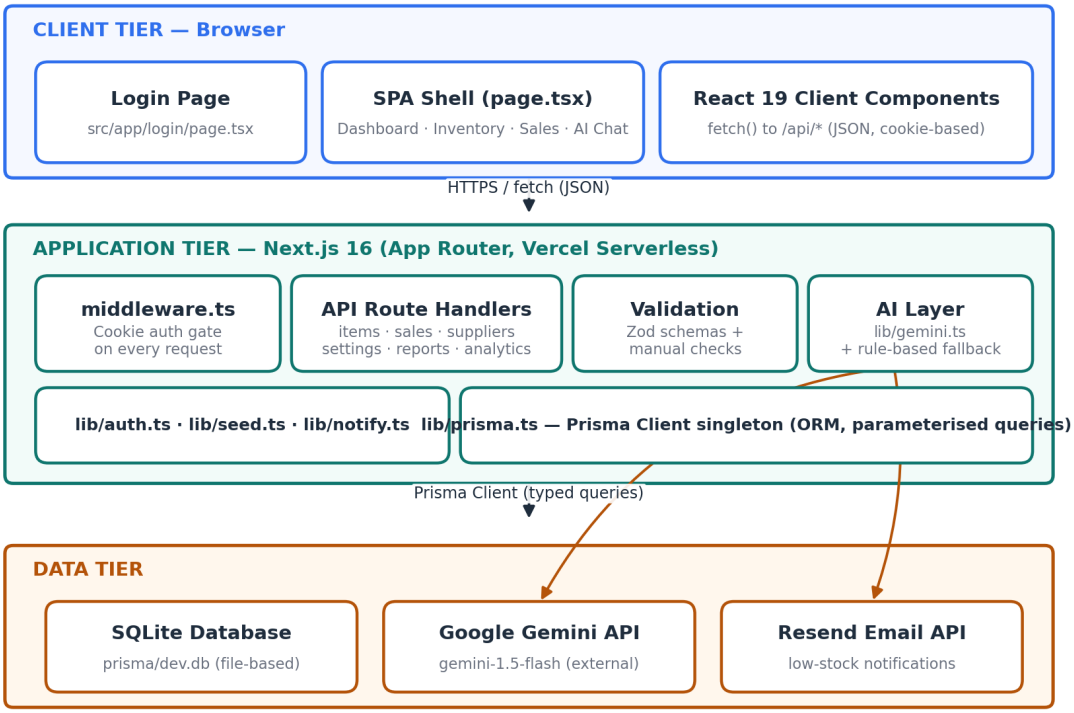

Figure A1. StockKeep three-tier architecture. Source: `architecture_diagram.png`.

16. Use Case Analysis

The system has one human actor (Shop Staff, holding the shared passcode) and one external system actor (Google Gemini). Every use case other than "Log in with passcode" requires the login use case first, enforced in code by `src/middleware.ts` rather than only by UI convention — confirmed live in `Testing_Report.pdf` (TC-API-01–02, TC-API-23). See **Figure C1**.

Figure C1 — Use Case Diagram — StockKeep Inventory Management System

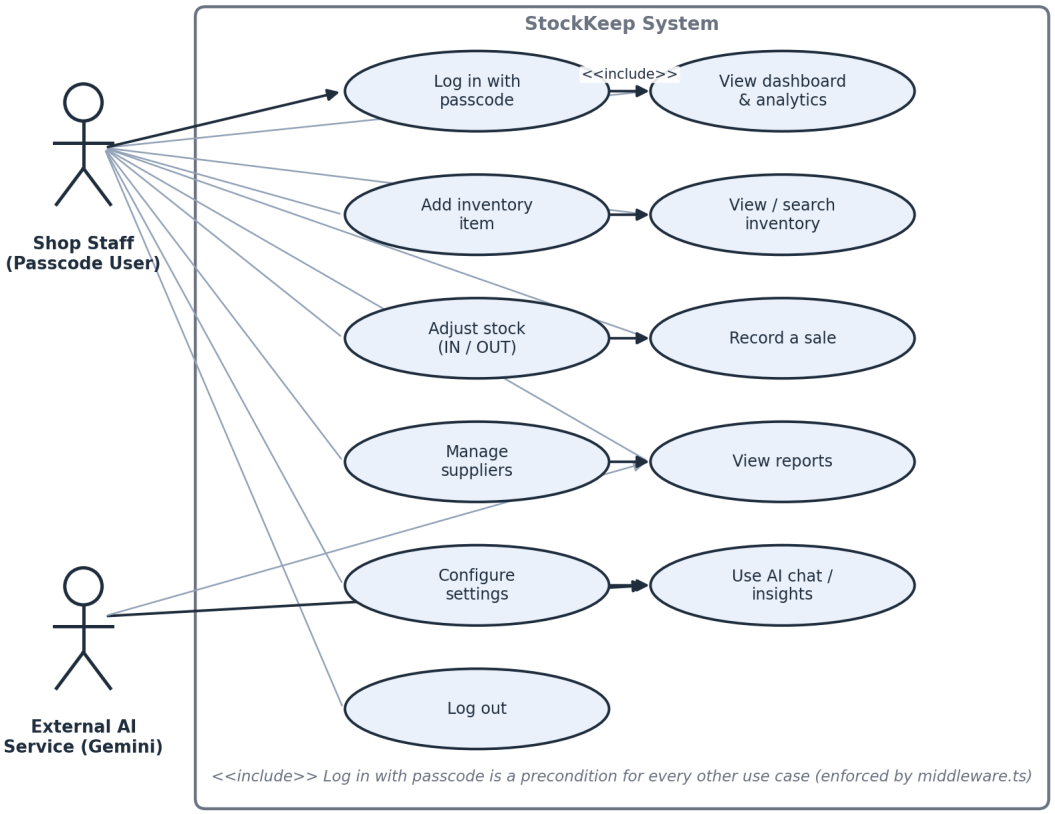

Figure C1. Use case diagram. Source: `use_case_diagram.png`.

17. Database / ER Design

The data model has five Prisma models: `Item`, `StockMovement`, `Sale`, `Supplier`, `Settings` (a singleton configuration row). `Item` is the hub entity, related to `Supplier` (many-to-one), `StockMovement` (one-to-many, cascade delete), and `Sale` (one-to-many, cascade delete). See **Figure D1**, generated directly from `prisma/schema.prisma`.

Figure D1 — Entity-Relationship Diagram (prisma/schema.prisma)

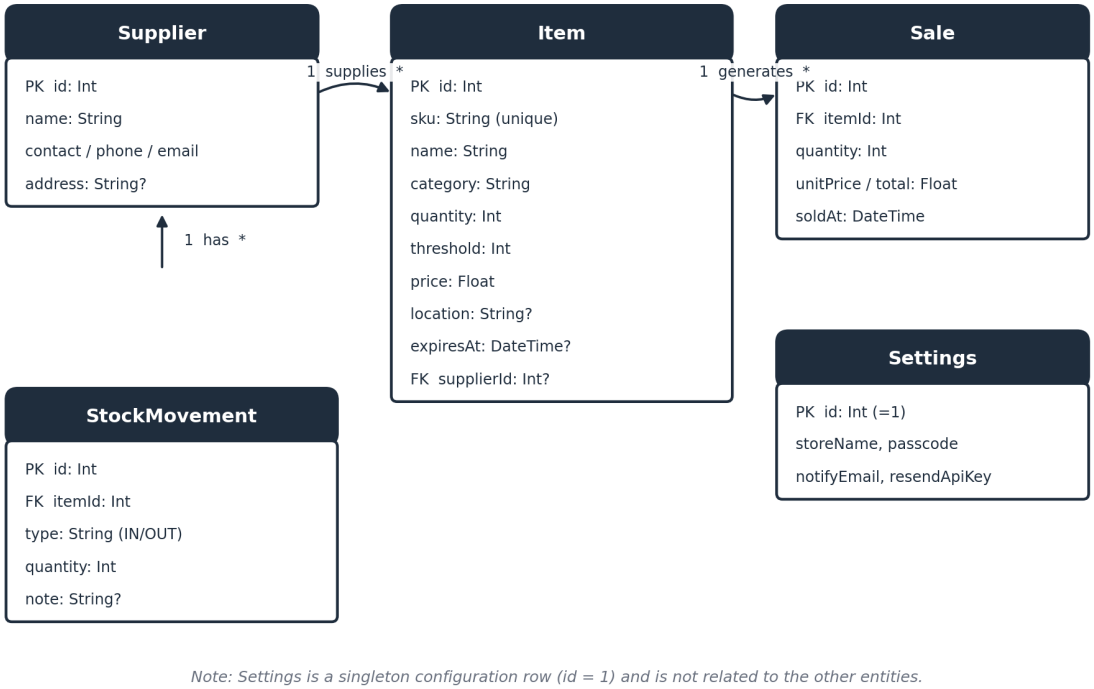

Figure D1. Entity-relationship diagram. Source: `er_diagram.png`.

18. UML Diagrams

Three further diagram types complete the UML view of the system: sequence diagrams for the three most significant runtime flows, an activity diagram for the stock-adjustment business rule, and a component diagram of the module structure. All were generated directly from the source files named in each caption, not from a generic template.

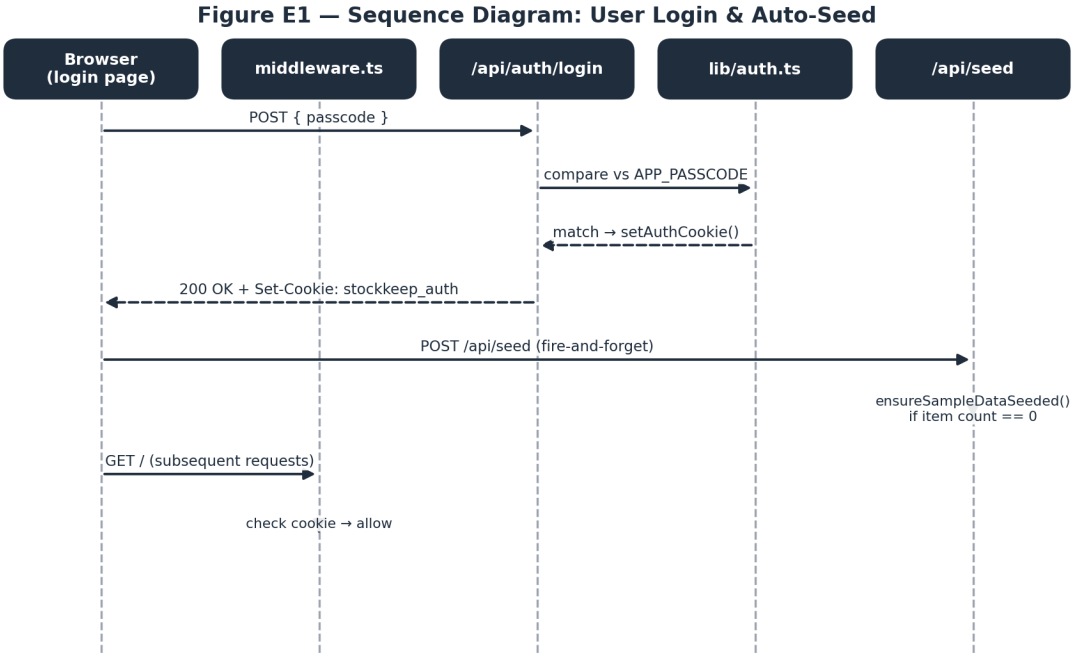

Figure E1. Sequence diagram — user login and auto-seed. Source: `sequence_login.png`.

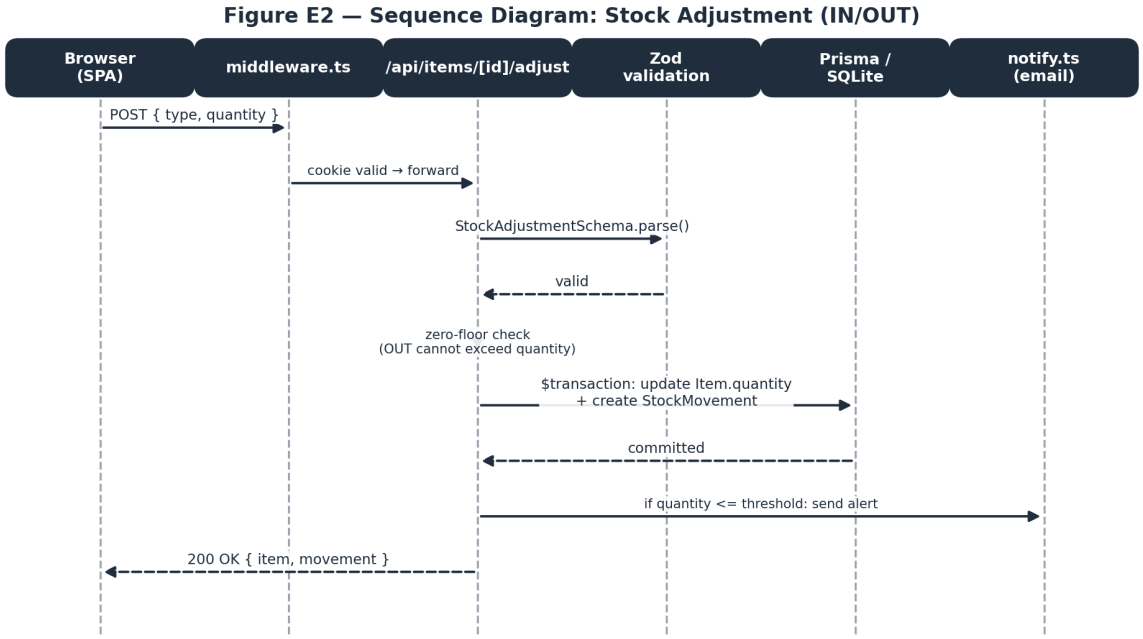

Figure E2. Sequence diagram — stock adjustment (IN/OUT). Source: `sequence_stock_adjustment.png`.

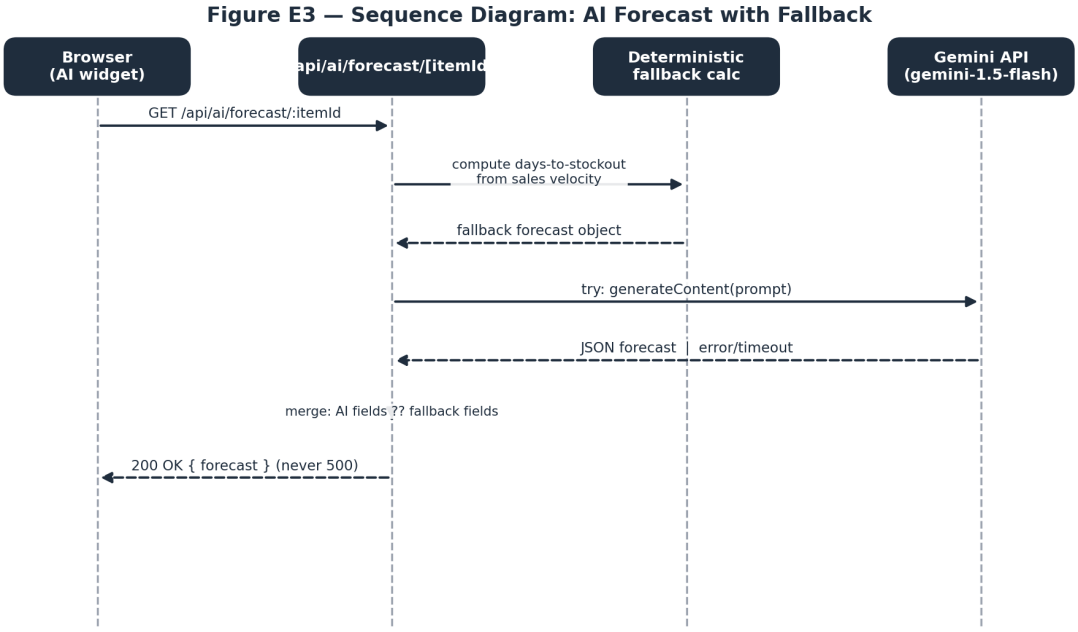

Figure E3. Sequence diagram — AI forecast with fallback. Source: `sequence_ai_forecast.png`.

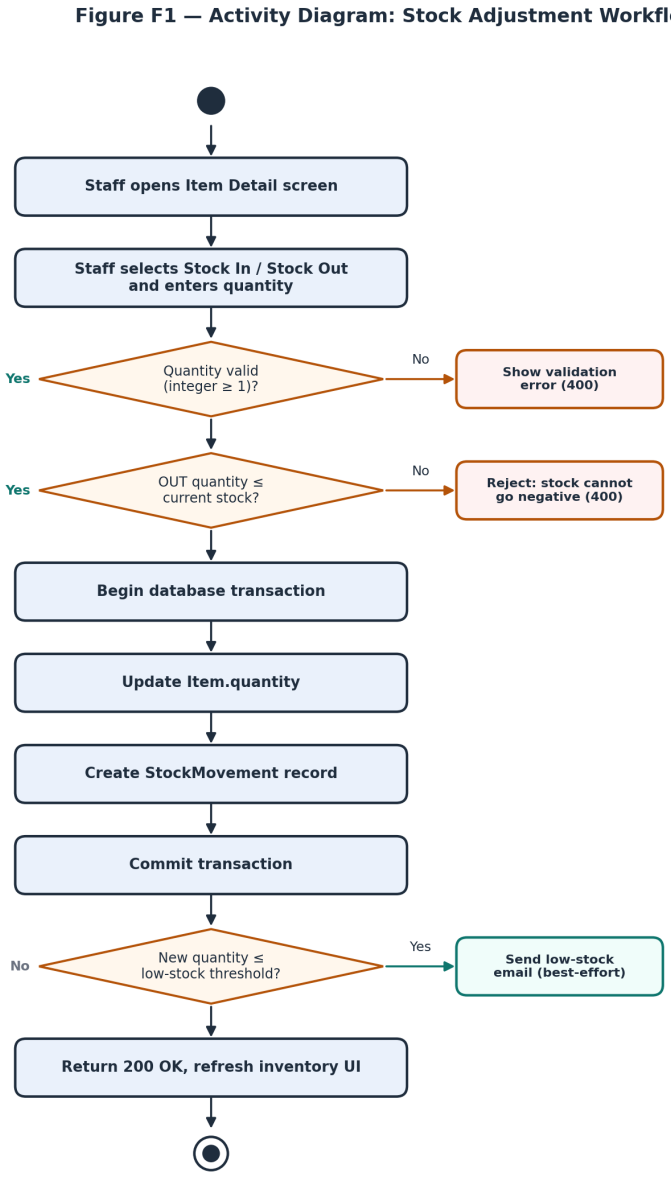

Figure F1. Activity diagram — stock adjustment workflow. Source: `activity_stock_adjustment.png`.

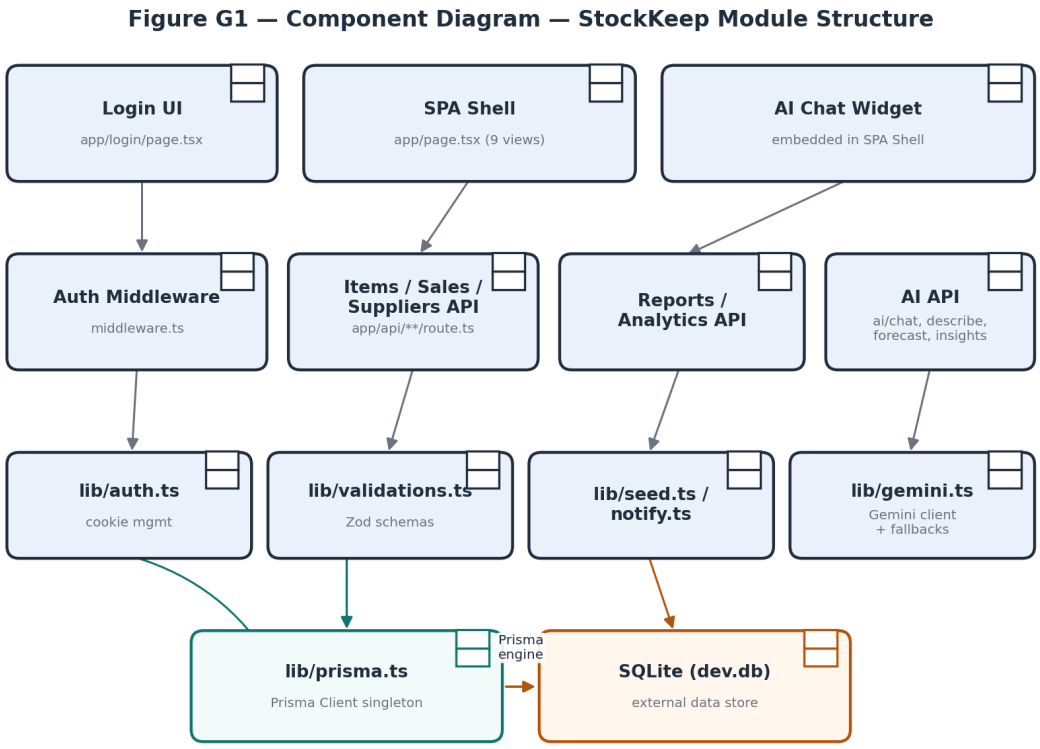

Figure G1. Component diagram — module structure. Source: `component_diagram.png`.

19. Implementation

StockKeep is implemented in TypeScript throughout (strict mode enabled — `tsconfig.json`), using Next.js 16.3.0, React 19.2.8, Prisma 5.22.0, Tailwind CSS 4, and `@google/generative-ai` 0.24.1. The following three sections detail the frontend, backend, and database layers respectively; Sections 23–25 cover authentication, validation, and security specifically.

20. Frontend

The entire UI is implemented as two App Router pages: `src/app/login/page.tsx` (the passcode screen) and a single large client component, `src/app/page.tsx` (1,921 lines), which implements nine tab-based views — Dashboard, Inventory, Low Stock, Sales, Suppliers, Reports, Settings, Add Item, and Item Detail — as conditionally rendered sections of one component, plus an embedded AI chat widget. Styling is done with Tailwind CSS 4 utility classes. **Table 6** in Section 21 below summarises the API surface the frontend consumes. The monolithic-component structure is recorded as technical debt (TD-09) rather than presented as an ideal design; see `Technical_Debt_Plan.pdf`.

21. Backend

The backend is implemented entirely as Next.js API route handlers under `src/app/api/`, each exporting HTTP-method-named functions (`GET`, `POST`, `PUT`, `DELETE`) per Next.js App Router convention. Business logic — the stock zero-floor rule, sale/stock reconciliation, SKU auto-generation, dead-stock detection, AI fallback computation — lives directly in these route handlers and in `src/lib/` helper modules (`auth.ts`, `validations.ts`, `seed.ts`, `notify.ts`, `gemini.ts`, `prisma.ts`).

Table 6 — API Route Surface

Route	Methods	Purpose
`/api/auth/login`, `/api/auth/logout`	POST	Session management
`/api/items`, `/api/items/[id]`	GET/POST/PUT/DELETE	Inventory CRUD
`/api/items/[id]/adjust`	POST	Stock-In/Stock-Out
`/api/sales`	GET/POST	Sales recording
`/api/suppliers`, `/api/suppliers/[id]`	GET/POST/PUT/DELETE	Supplier CRUD
`/api/settings`	GET/PUT	Store configuration
`/api/reports/summary`, `/api/analytics`	GET	Reporting/analytics
`/api/seed`	GET/POST	Sample data bootstrap
`/api/ai/chat`, `/describe`, `/forecast/[itemId]`, `/insights`	POST/GET	AI features with fallback

22. Database

SQLite (`prisma/dev.db`), accessed exclusively through Prisma Client (`src/lib/prisma.ts`, a singleton to avoid connection exhaustion in the Next.js dev/hot-reload environment). Five models as shown in **Figure D1** (Section 17). Multi-row writes that must be atomic (stock adjustment, sale recording) use Prisma's `$transaction` API, confirmed correct under live testing (`Testing_Report.pdf`, TC-API-10, TC-API-11, TC-API-13).

23. Authentication & Authorisation

A single shared passcode (`APP_PASSCODE` environment variable, default `1234`) gates the entire application via an HTTP-only session cookie (`stockkeep_auth`, 7-day expiry, `SameSite=Lax`) checked by `src/middleware.ts` on every request except `/login`, `/api/auth/*`, and `/api/seed`. There is **no authorisation/role separation** — every authenticated session has identical, full access. Two authentication-related gaps were found and confirmed by live testing: the Settings-page passcode field does not actually control login (TD-01), and `/api/seed` is reachable without authentication at all (TD-02, confirmed exploitable in `Testing_Report.pdf` TC-API-24). Full detail in `SRS.pdf` Section 6.

24. Validation

Zod schemas (`src/lib/validations.ts`) validate item creation/update and stock adjustments; sales and supplier routes use simpler manual checks; the settings route performs no validation at all. This inconsistency is intentional-but-undesirable technical debt, not an oversight discovered late — see `Technical_Debt_Plan.pdf`. Live testing found that invalid input is *correctly rejected before reaching the database* in every case tested, but two of those rejection paths return an unhelpful HTTP 500 instead of a clean 400 due to a Next.js dev-mode error-formatting fault (DEF-01, `Testing_Report.pdf`).

25. Security

No raw SQL exists in the codebase (Prisma's parameterised queries only, confirmed by code search). Secrets are loaded from environment variables and `.env*` is git-ignored, with one exception: the Resend email API key is stored in the `Settings` database table and returned in plaintext by `GET /api/settings` (TD-06, confirmed live). No security response headers (CSP/HSTS/X-Frame-Options), no CSRF tokens, and no login rate limiting are configured — the last of these was confirmed live by five rapid failed login attempts all succeeding immediately with no lockout (`Testing_Report.pdf`, TC-API-25). Full detail in `SRS.pdf` Section 7 and `Technical_Debt_Plan.pdf`.

26. Testing

Testing combined 26 live-executed black-box API test cases (`curl` against a running local instance), code review, and explicit "not verified" labelling for anything neither executed nor determinable from source (UI rendering, load testing, cross-device usability). The full strategy, environment, test cases, and results are in the standalone **`Testing_Report.pdf`**; raw evidence logs are in `Supporting_Files/Test_Evidence/`.

27. Test Results

Table 7 — Testing Summary

Metric	Result
API-level tests executed	26
Clean passes	21
Functional failures	2 (DEF-01, appearing in TC-API-06 and TC-API-12)
Confirmed security gaps	2 (SEC-04 via TC-API-24, SEC-05 via TC-API-25)
Partial results	1 (TC-API-26 — cookie attributes correct, security headers absent)
Automated test coverage	0% (no test suite exists in the repository)
UI / UAT / load testing	Not executed — marked "Not verified — execution required by student"

Full per-test detail, defect descriptions (DEF-01 through DEF-05), and the requirements-to-test traceability matrix are in `Testing_Report.pdf`.

28. Technical Debt

Fifteen technical debt items (TD-01 through TD-15) were identified through direct code inspection and live testing, spanning authentication design, an unauthenticated destructive endpoint, missing rate limiting, missing security headers, a plaintext secret exposure, zero automated test coverage, a production-unsuitable database choice for the stated deployment target, a monolithic frontend component, missing pagination, an unreliable in-memory cache, silent error handling, and a misconfigured/unavailable AI model. Each is classified **Critical / Immediate**, **Scheduled for Future Resolution**, or **Acceptable Temporarily**.

Table 8 — Technical Debt Register (abridged; full register in `Technical_Debt_Plan.pdf`)

ID	Item	Classification
TD-01	Settings passcode disconnected from login	Critical / Immediate
TD-02	Unauthenticated destructive seed endpoint	Critical / Immediate
TD-03	No login rate limiting	Critical / Immediate
TD-06	Plaintext Resend API key exposure	Critical / Immediate
TD-08	SQLite on ephemeral serverless target	Critical / Immediate
TD-13	Gemini model unavailable (404) — AI always on fallback	Critical / Immediate
TD-04, TD-05, TD-07, TD-12	CSRF, security headers, no tests, silent errors	Scheduled for Future Resolution
TD-09, TD-10	Monolithic frontend, no pagination	Scheduled for Future Resolution
TD-11, TD-15	In-memory cache, loose error typing	Acceptable Temporarily

29. Technical Debt Repayment Plan

Repayment is sequenced Immediate → Version 1.1 → Version 2.0 → Long-Term, detailed in full in `Technical_Debt_Plan.pdf`. In summary: Immediate work removes trust-breaking risks (auth correctness, data-destruction exposure, secret exposure, persistence risk, AI misconfiguration); Version 1.1 converts the manual security/testing findings from this documentation pass into durable automated safeguards (CSRF, headers, an automated test suite seeded from the 26 documented test cases); Version 2.0 addresses structural debt (frontend decomposition, query-level pagination); Long-Term items depend on real usage data and product direction (multi-user roles, database re-evaluation).

30. Deployment

StockKeep targets Vercel's zero-configuration Next.js hosting. `package.json`'s `build` script runs `prisma generate && next build --webpack`, and `postinstall` also runs `prisma generate`, both addressing a previously-fixed Vercel/Prisma build failure (commit `d3d3ede`). AI and data-dependent routes are marked `export const dynamic = 'force-dynamic'` so they are not statically cached. On first request to an empty database, `ensureSampleDataSeeded()` populates a full demo dataset automatically (commit `c44398c`) — a pragmatic compensation for SQLite's poor fit with serverless persistence (see TD-08). No `vercel.json` and no CI/CD pipeline (`.github/workflows`) exist; deployment is manual. Live deployment URL, environment variables, and access credentials for this specific submission are recorded in the standalone **`Deployment_and_Source_Links.txt`**.

31. User Manual

Full step-by-step usage instructions for every screen (login, dashboard, inventory, stock-in/out, sales, suppliers, reports, settings, AI assistant, logout, troubleshooting) are provided in the standalone **`User_Manual.pdf`**, written for a non-technical shop-staff audience and cross-referencing the same requirement IDs and defect IDs used throughout this package.

32. Maintenance Strategy

1. Treat `SRS.pdf`, `Testing_Report.pdf`, and `Technical_Debt_Plan.pdf` as living documents — update them in the same commit/PR as any change that affects a requirement, test, or debt item they describe.
2. Before merging any change to `src/lib/validations.ts`, `middleware.ts`, or any `api/items|sales|suppliers` route, re-run the 26 test cases in `Testing_Report.pdf` Section 5 (manually until TD-07 is resolved, then via the automated suite).
3. Re-verify the Gemini model configuration (TD-13) periodically, since third-party model availability is outside this project's control and can silently regress feature quality without an outright failure.
4. Review the Technical Debt Register at the start of each new development cycle and re-prioritise based on real incident/usage data, not only the priorities set at initial authoring time.

33. Future Evolution

Beyond the Technical Debt Repayment Plan (Version 2.0 and Long-Term items), natural next features include: barcode/QR scanning for faster stock-in, multi-branch/multi-store support, individual staff accounts with audit trails (a natural extension once TD-01's authentication rework is done), purchase-order generation direct from the reorder-forecast data already computed by `/api/ai/forecast`, and a real analytics warehouse if reporting needs outgrow live SQLite aggregation queries.

34. Limitations

- Single shared passcode, no per-staff identity or audit trail of *who* performed an action (only *that* it happened).
- No automated test suite; correctness confidence rests on the 26 manually-executed tests in this package plus ongoing manual testing.
- SQLite is not a safe long-term production database for the stated serverless deployment target (TD-08).
- AI features are currently running on fallback logic only in this environment due to a model-availability issue (TD-13) — the fallback logic itself is correct and was verified live, but genuine AI-generated responses were not observed during this testing pass.
- No UI-level, load, or usability testing was performed as part of this documentation (no browser automation was available in the environment used to prepare it) — see `Testing_Report.pdf` for the exact scope executed.
- No real screenshots could be captured for the same reason; the User Manual uses labelled placeholders.

35. Conclusion

StockKeep delivers a working, coherent single-shop inventory and stock-tracking system covering the full stock lifecycle, sales reconciliation, reporting, and an AI assistant layer engineered to degrade gracefully rather than fail. Direct source inspection and live API testing confirm that the core business rules (non-negative stock, atomic transactions, sale-stock reconciliation, route-level authentication) work correctly as implemented. The same process surfaced five real defects and fifteen technical debt items, all documented here with concrete remediation steps rather than deferred silently — which is, itself, evidence of a system whose actual behaviour is now well understood rather than merely assumed.

36. References

1. Next.js Documentation — https://nextjs.org/docs
2. Prisma Documentation — https://www.prisma.io/docs
3. React Documentation — https://react.dev
4. Zod Documentation — https://zod.dev
5. Google Generative AI SDK (`@google/generative-ai`) — https://ai.google.dev
6. Tailwind CSS Documentation — https://tailwindcss.com/docs
7. Vercel Deployment Documentation — https://vercel.com/docs
8. StockKeep source repository (this project) — see `Deployment_and_Source_Links.txt` for the repository URL.

37. Final Compliance Checklist

Table 9 — Final Compliance Checklist

Item	Status
All 37 required sections present in this document	Complete
SRS.pdf, Testing_Report.pdf, Technical_Debt_Plan.pdf, User_Manual.pdf produced as standalone documents	Complete
Deployment_and_Source_Links.txt produced	Complete
Supporting_Files/ populated with real diagrams generated from source	Complete
Diagrams: Architecture, ER, Use Case, 3× Sequence, Activity, Component	Complete (8 diagrams)
Live-executed test evidence (not fabricated)	Complete — `Supporting_Files/Test_Evidence/`
Requirements traceability matrix	Complete
No fabricated screenshots	Complete — labelled placeholders used instead
No fabricated credentials	Complete — all left as `[TO BE PROVIDED]`
No fabricated test results	Complete — every result is either live-executed, code-review-labelled, or marked not verified
Student-identifying fields (name, ID, institution, supervisor)	**Requires student input**
Live deployment URL and test/admin credentials	**Requires student input**
Real application screenshots	**Requires student action** (capture from a running instance)
UI, UAT, load/performance testing	**Requires student action**
Production-build re-verification of DEF-01	**Requires student action**

See `FINAL_SUBMISSION_READINESS_REPORT.md` for the complete, itemised readiness assessment.